Core DevOps Viva Questions

04/09/2026 14:01:39

please provide all the viva questions from core devops inckuding all the neccesity i…

Absolutely — I'll organize this as a complete Core DevOps Viva Master Question Ba…

not just simple in one chat there should be total overview about that what is it and…

Perfect — that's a much better way to prepare for viva. We'll do this as a full…

yes

Great — please type: NEXT and I'll begin Topic 2: Complete SDLC Overview in full…

Next

TOPIC 2: COMPLETE SDLC OVERVIEW (Software Development Life Cycle – Full…

NEXT

TOPIC 3: AGILE & SCRUM COMPLETE OVERVIEW (Complete Theory for DevOps Vi…

NEXT

TOPIC 4: GIT & GITHUB COMPLETE OVERVIEW (Full DevOps Viva Theory + Practi…

NEXT

TOPIC 5: CI/CD COMPLETE OVERVIEW (Continuous Integration / Continuous…

NEXT

TOPIC 6: JENKINS COMPLETE OVERVIEW (Full DevOps Viva Theory + Real-Time…

NEXT

TOPIC 7: DOCKER COMPLETE OVERVIEW (Full DevOps Viva Theory + Practical…

NEXT

TOPIC 8: KUBERNETES COMPLETE OVERVIEW (Full DevOps Viva Theory + Real-…

NEXT

TOPIC 9: SELENIUM COMPLETE OVERVIEW (Full DevOps Viva Theory + Testing…

NEXT

TOPIC 10: LINUX FOR DEVOPS COMPLETE OVERVIEW (Full DevOps Viva Theory +…

NEXT

TOPIC 11: CLOUD COMPUTING FOR DEVOPS COMPLETE OVERVIEW (Full DevOps…

please provide all the viva questions from core devops inckuding all the neccesity in it to all the theroy part like sdlc,scrum etc many morr theory and on git,github and some more tools used for devops and the docker,kubernetes,jenkins,selenium etc please give all the odt important questions that evry student must know about it and be careful and give neatly can you give chat wuse no problem

09/04/2026, 13:52:12

Absolutely — I'll organize this as a complete **Core DevOps Viva Master Question Bank** covering all major theory areas and tools in a clean chat-wise format.

This includes:

1. DevOps Fundamentals
2. SDLC Models
3. Agile & Scrum
4. Git & GitHub
5. CI/CD Concepts
6. Jenkins
7. Docker
8. Kubernetes
9. Selenium
10. Linux Basics for DevOps
11. Cloud Basics
12. Monitoring Tools
13. Important Scenario-Based Viva Questions
14. Must-Know Interview/Viva Questions

PART 1: DEVOPS FUNDAMENTALS

---

1. What is DevOps?

DevOps is a combination of:

- Development (Dev)
- Operations (Ops)

It is a methodology that improves collaboration between development and operations teams to automate software delivery.

---

2. Why DevOps is needed?

Because traditional software development had:

- Slow delivery
- Manual deployment errors
- Poor communication

DevOps solves this by:

- Automation
- Faster releases
- Continuous feedback

---

3. What are DevOps goals?

Main goals:

- Faster delivery
- Better collaboration
- Automation
- Continuous integration
- Reliable deployment

---

4. What is DevOps lifecycle?

Stages:

1. Plan
2. Develop
3. Build

4. Test
5. Release
6. Deploy
7. Operate
8. Monitor

---

5. Difference between Agile and DevOps?

Agile	DevOps
Focus on development	Focus on development + operations
Faster coding	Faster coding + deployment
Sprint based	Continuous delivery

PART 2: SDLC

---

6. What is SDLC?

Software Development Life Cycle is process of software creation.

Phases:

1. Requirement Gathering
2. Design
3. Development
4. Testing
5. Deployment
6. Maintenance

---

7. Types of SDLC models?

Important models:

- Waterfall

- V-Model
- Spiral
- Agile
- Incremental

---

8. Difference between Waterfall and Agile?

Waterfall:

- Sequential

Agile:

- Iterative
- Flexible

PART 3: AGILE & SCRUM

---

9. What is Agile?

Agile is software development approach based on iterative progress.

---

10. What is Scrum?

Scrum is Agile framework where work is divided into sprints.

---

11. What is Sprint?

Sprint = Fixed time period (usually 2 weeks) to complete tasks.

---

12. Scrum Roles?

1. Product Owner
2. Scrum Master

3. Development Team

---

13. Scrum Artifacts?

- Product backlog
- Sprint backlog
- Increment

---

14. Daily Scrum means?

Daily 15-minute stand-up meeting.

PART 4: GIT & GITHUB

---

15. What is Git?

Git is distributed version control system.

---

16. What is GitHub?

GitHub is cloud platform to host Git repositories.

GitHub

---

17. Difference between Git and GitHub?

Git = Tool

GitHub = Platform

---

18. Important Git commands?

```
git init
git clone
git add .
git commit -m "msg"
git push
git pull
git status
git log
```

---

19. What is branch in Git?

Separate development line.

Example:

```
git branch feature1
```

---

20. What is merge conflict?

Occurs when same file edited differently.

---

21. What is fork?

Copy of repository into another account.

---

22. What is pull request?

Request to merge code changes.

PART 5: CI/CD

---

23. What is CI?

Continuous Integration = Automatically integrating code changes.

---

24. What is CD?

Continuous Delivery / Deployment = Automatic software release.

---

25. Difference CI vs CD?

CI = Build + Test

CD = Deploy

PART 6: JENKINS

---

26. What is Jenkins?

Jenkins is automation server used for CI/CD.

Jenkins

---

27. What is Jenkins Pipeline?

Automated sequence of build steps.

---

28. What is Jenkinsfile?

File containing pipeline code.

Example:

```
pipeline {
 agent any
 stages {
 stage('Build') {
```

```
  steps {
  echo 'Building...'
  }
  }
 }
}
```

---

29. Types of Pipeline?

- Declarative
- Scripted

---

30. Jenkins Master-Agent Architecture?

Master controls jobs

Agent executes jobs

PART 7: DOCKER

---

31. What is Docker?

Docker is containerization platform.

Docker

---

32. What is container?

Lightweight isolated package containing app + dependencies.

---

33. Difference VM vs Container?

VM:

- Full OS

Container:

- Shared OS kernel

---

34. What is Docker Image?

Blueprint of container.

---

35. Dockerfile means?

Instructions to build image.

Example:

```
FROM node:18
COPY . /app
WORKDIR /app
RUN npm install
CMD ["npm","start"]
```

---

36. Docker commands?

```
docker build
docker run
docker ps
docker images
docker stop
docker rm
```

---

37. What is Docker Hub?

Cloud registry for Docker images.

Docker

PART 8: KUBERNETES

---

38. What is Kubernetes?

Container orchestration tool.

Kubernetes

---

39. Why Kubernetes needed?

To manage multiple containers automatically.

---

40. What is Pod?

Smallest deployable unit.

---

41. What is Deployment?

Manages pod replicas.

---

42. What is Service?

Exposes pods to network.

---

43. Types of Services?

- ClusterIP
- NodePort
- LoadBalancer

---

44. What is ReplicaSet?

Maintains desired pod count.

45. What is Minikube?

Local Kubernetes cluster.

Minikube

PART 9: SELENIUM

---

46. What is Selenium?

Automation testing tool for web applications.

Selenium

---

47. Selenium components?

- WebDriver
- IDE
- Grid

---

48. What is WebDriver?

Controls browser automatically.

PART 10: LINUX FOR DEVOPS

---

49. Common Linux commands?

```
ls
pwd
cd
mkdir
```

```
rm
cp
mv
chmod
ps
top
grep
```

---

50. chmod means?

Change file permissions.

---

51. What is shell scripting?

Automating Linux commands using script.

PART 11: CLOUD BASICS

---

52. What is cloud computing?

Using computing resources over internet.

---

53. Types of cloud?

- Public
- Private
- Hybrid

---

54. Popular cloud providers?

- Amazon Web Services
- Microsoft Azure

- Google Cloud

PART 12: MONITORING TOOLS

---

55. What is monitoring in DevOps?

Tracking system performance.

---

56. Popular monitoring tools?

- Prometheus
- Grafana
- Nagios

PART 13: SCENARIO QUESTIONS

---

57. If Jenkins build fails what do you check?

Check:

- Logs
- Syntax errors
- Dependencies
- Network issues

---

58. Docker container exits immediately why?

Possible reasons:

- Main process finished
- CMD error

---

59. Pod in CrashLoopBackOff means?

Pod repeatedly crashing.

---

60. Git merge conflict how resolve?

Edit conflicting file manually and commit again.

PART 14: MOST IMPORTANT MUST-KNOW QUESTIONS

These are asked almost always:

Top 20 Must Prepare:

1. What is DevOps?
2. Explain CI/CD.
3. What is Git?
4. Difference Git vs GitHub?
5. What is Jenkins?
6. What is pipeline?
7. What is Docker?
8. Image vs Container?
9. What is Kubernetes?
10. Pod vs Container?
11. Deployment vs Service?
12. What is Scrum?
13. Sprint meaning?
14. SDLC phases?
15. Agile vs Waterfall?
16. What is Dockerfile?
17. What is Jenkinsfile?
18. What is Minikube?
19. What is Selenium?

20. Explain DevOps lifecycle.

BONUS: EXTERNAL EXAMINER FAVORITE QUESTIONS

Very Frequently Asked:

- Why DevOps over traditional model?
- Explain real-time CI/CD pipeline.
- How Jenkins integrates with GitHub?
- Why Kubernetes better than Docker alone?
- Difference between deployment and replica set?

FINAL TIP FOR VIVA

If asked unknown question:

Say:

"I am not fully sure, but based on my understanding..."

This creates positive impression.

Would you also like:

➜ Advanced DevOps Viva Questions (Expert Level)

09/04/2026, 13:52:13

or

➜ Real-Time Project-Based DevOps Viva Questions?

not just simple in one chat there should be total overview about that what is it and all about that catergory
when i say next you cintinue withother tpics

09/04/2026, 13:53:13

Perfect — that's a much better way to prepare for viva.

We'll do this as a **full masterclass-style DevOps viva preparation series**, where each topic is covered in deep detail:

- Definition
- Why it is needed
- Architecture
- Components
- Real-time usage
- Important viva questions
- Interview tricky questions
- Examples

We will go **one major topic at a time**, and when you say **"next"**, I will continue with the next topic.

TOPIC 1: COMPLETE DEVOPS OVERVIEW

---

1. INTRODUCTION TO DEVOPS

DevOps is one of the most important concepts in modern software engineering.

The word **DevOps = Development + Operations**

It is a software development culture that combines:

- Developers who write code
- Operations team who deploy and maintain software

Before DevOps existed:

Earlier:

- Developers only developed software
- Operations team only deployed software

Problem:

- Poor communication
- Slow releases
- Frequent deployment failures

Example:

Developer says:

> *"Code works on my machine."*

Operations says:

> *"It fails on server."*

This conflict caused delays.

DevOps solves this problem by:

Creating collaboration between:

Development Team + Testing Team + Operations Team

So software becomes:

✔️ Faster
✔️ Reliable
✔️ Automated
✔️ Scalable

---

2. WHY DEVOPS IS NEEDED?

Main reasons:

A. Faster Software Delivery

Companies release updates quickly.

Example:

Apps like:

- Instagram
- WhatsApp
  update frequently.

B. Automation

Manual work reduced.

Example:

Instead of manually deploying:

Jenkins deploys automatically.

C. Fewer Errors

Automation reduces human mistakes.

D. Continuous Monitoring

Problems detected early.

---

3. DEVOPS DEFINITION

Official Definition:

> *DevOps is a set of practices that automates and integrates software development and IT operations.*

---

4. MAIN OBJECTIVES OF DEVOPS

Goal 1: Continuous Integration

Code merged regularly.

Goal 2: Continuous Delivery

Software always ready for release.

Goal 3: Continuous Deployment

Automatic deployment without manual approval.

Goal 4: Infrastructure Automation

Servers configured automatically.

---

5. DEVOPS LIFECYCLE

DevOps lifecycle has 8 phases:

Phase 1: PLAN

Requirement gathering.

Tools:

- Jira
- Trello

Phase 2: DEVELOP

Writing code.

Tools:

- Git
- GitHub

GitHub

Phase 3: BUILD

Compile application.

Tools:

- Maven
- Gradle

Phase 4: TEST

Testing software.

Tools:

- Selenium

Selenium

Phase 5: RELEASE

Prepare release version.

Phase 6: DEPLOY

Deploy into production.

Tools:

- Jenkins
- Docker

Jenkins

Docker

Phase 7: OPERATE

Run software in servers.

Phase 8: MONITOR

Track performance.

Tools:

- Prometheus
- Grafana

---

6. DEVOPS ARCHITECTURE

Basic architecture:

```
Developer → Git → Jenkins → Docker → Kubernetes → Production Server →
Monitoring
```

Explanation:

1. Developer writes code
2. Pushes to Git
3. Jenkins builds project
4. Docker packages app
5. Kubernetes manages containers
6. Production server runs app
7. Monitoring tracks health

---

7. CORE PRINCIPLES OF DEVOPS

Principle 1: Collaboration

Teams work together.

Principle 2: Automation

Everything automated.

Principle 3: Continuous Feedback

Immediate issue detection.

Principle 4: Continuous Improvement

Always optimize process.

---

8. BENEFITS OF DEVOPS

Faster Releases

Deployment becomes rapid.

Improved Quality

More testing = fewer bugs.

Better Stability

Reliable systems.

Reduced Costs

Less manual effort.

9. DEVOPS VS TRADITIONAL MODEL

Traditional	DevOps
Separate teams	Collaborative teams
Slow release	Fast release
Manual deployment	Automated deployment
High failure rate	Lower failure rate

10. POPULAR DEVOPS TOOLS

Important tools:

Category	Tool
Version Control	Git
CI/CD	Jenkins
Containerization	Docker
Orchestration	Kubernetes
Testing	Selenium
Monitoring	Prometheus

---

11. REAL-TIME EXAMPLE OF DEVOPS

Suppose company develops shopping app.

Without DevOps:

- Deployment takes 2 weeks

With DevOps:

- Deployment takes 10 minutes

Example flow:

1. Developer commits code
2. Git triggers Jenkins
3. Jenkins runs tests
4. Docker builds image
5. Kubernetes deploys app

---

12. COMMON DEVOPS CHALLENGES

Challenge 1: Tool Complexity

Many tools to learn.

Challenge 2: Security Risks

Automation must be secured.

Challenge 3: Cultural Resistance

Teams resist change.

---

13. IMPORTANT VIVA QUESTIONS ON DEVOPS

Q1. What is DevOps?

DevOps combines development and operations to improve software delivery.

Q2. Why DevOps important?

Because it speeds up delivery and improves collaboration.

Q3. What are DevOps phases?

Plan, Develop, Build, Test, Release, Deploy, Operate, Monitor.

Q4. Difference between Agile and DevOps?

Agile improves development process.
DevOps improves development + deployment.

Q5. Name major DevOps tools.

Git, Jenkins, Docker, Kubernetes, Selenium.

---

14. TRICKY EXAMINER QUESTIONS

"Is DevOps a tool?"

Answer:

No, DevOps is not a tool.

It is a culture/process.

"Can DevOps exist without automation?"

Possible, but ineffective.

Automation is core strength.

"Who uses DevOps?"

Software companies:

- Amazon
- Google
- Netflix

---

15. SUMMARY TO REMEMBER IN ONE LINE

DevOps = Culture + Automation + Collaboration + Continuous Delivery

---

When you're ready, say:

NEXT

Then I will continue with:

TOPIC 2: SDLC COMPLETE OVERVIEW

Including:

- All SDLC models
- Waterfall
- Spiral
- Agile

Great, please type:

- Comparison charts
- Viva questions in depth

NEXT

and I'll begin **Topic 2: Complete SDLC Overview** in full detail.

09/04/2026, 13:53:52

yes

09/04/2026, 13:53:50

Next

09/04/2026, 13:54:19

TOPIC 2: COMPLETE SDLC OVERVIEW

(Software Development Life Cycle – Full Theory for DevOps Viva)

---

1. WHAT IS SDLC?

SDLC stands for:

Software Development Life Cycle

It is a structured process used to design, develop, test, and maintain software systems.

In simple words:

> *SDLC is the step-by-step method followed to build software properly.*

Real Meaning:

Whenever a company creates software like:

- WhatsApp
- Instagram
- Microsoft Word

They follow SDLC phases.

2. WHY SDLC IS IMPORTANT?

Without SDLC:

- No planning
- Confusion in development
- High bugs
- Project delays

With SDLC:

✔ Clear roadmap
✔ Better quality
✔ Cost control
✔ Time management

---

3. MAIN PURPOSE OF SDLC

Goal of SDLC:

"Deliver high-quality software within time and budget."

---

4. PHASES OF SDLC

There are 6 major phases:

PHASE 1: REQUIREMENT GATHERING

First step:

Client explains what software is needed.

Example:

Client says:

"I need online shopping website."

Collected requirements:

- Login page
- Cart system

- Payment gateway

Output:

Requirement Specification Document

PHASE 2: SYSTEM DESIGN

Architects create software blueprint.

Includes:

- Database design
- UI design
- Architecture planning

Example:
Designing:

- Homepage layout
- Login screen

PHASE 3: DEVELOPMENT (CODING)

Actual coding begins.

Developers use:

- Java
- Python
- PHP
- JavaScript

Example:
Writing login authentication code.

PHASE 4: TESTING

Check software for errors.

Types:

- Unit testing
- Integration testing
- System testing

Tools:

- Selenium

PHASE 5: DEPLOYMENT

Software released to users.

Example:

Website launched live.

PHASE 6: MAINTENANCE

After release:

Fix bugs and update software.

Example:

Adding new features later.

---

5. SDLC DIAGRAM FLOW

```
Requirement → Design → Development → Testing → Deployment → Maintenance
```

---

6. TYPES OF SDLC MODELS

Very important viva topic.

Main models:

1. Waterfall Model
2. V-Model
3. Spiral Model

4. Iterative Model
5. Incremental Model
6. Agile Model

---

7. WATERFALL MODEL

Oldest SDLC model.

Flow is sequential:

```
Phase 1 → Phase 2 → Phase 3 → Next
```

Cannot skip backward easily.

Advantages:

- Simple
- Easy to manage

Disadvantages:

- No flexibility
- Late bug detection

Best for:

Small fixed projects.

---

8. V-MODEL

Extension of Waterfall.

Each development phase has testing phase.

Example:

Requirement ↔ Acceptance Testing
Design ↔ System Testing

Advantage:

Testing starts early.

---

9. SPIRAL MODEL

Risk-driven model.

Used for large risky projects.

Repeated cycles:

1. Planning
2. Risk Analysis
3. Development
4. Evaluation

Best for:

Complex large systems.

Example:
Banking software.

---

10. ITERATIVE MODEL

Software built step by step.

Version 1 released first,
then improved repeatedly.

---

11. INCREMENTAL MODEL

System developed in small parts.

Example:

Phase 1:

Login module

Phase 2:

Payment module

Phase 3:

Reports module

---

12. AGILE MODEL

Most important modern model.

Development in short cycles.

Called:

Iterations / Sprints

Agile focuses on:

- Flexibility
- Fast updates
- Customer feedback

Example:

Apps like:

- Facebook
- Uber

use Agile.

---

13. WATERFALL VS AGILE

Waterfall	Agile
Sequential	Iterative
Rigid	Flexible
Late testing	Continuous testing
Slow updates	Fast updates

---

14. SDLC IN DEVOPS

Traditional SDLC:

Development ends before deployment.

DevOps SDLC:

Continuous cycle:

Build → Test → Deploy → Monitor repeatedly

DevOps improves SDLC by:

- Automation
- Faster deployment
- Continuous testing

---

15. REAL-TIME EXAMPLE

Suppose making food delivery app:

Requirement:

Need order system

Design:

UI screens planned

Development:

Code written

Testing:

Check payment bugs

Deployment:

Launch app

Maintenance:

Fix customer complaints

---

16. COMMON SDLC PROBLEMS

Problem 1:

Changing requirements

Problem 2:

Poor communication

Problem 3:

Late bug detection

Problem 4:

Budget overrun

---

17. IMPORTANT VIVA QUESTIONS

Q1: What is SDLC?

SDLC is structured process for software development.

Q2: Why SDLC needed?

To ensure software quality and proper planning.

Q3: Name SDLC phases.

1. Requirement Gathering
2. Design
3. Development
4. Testing
5. Deployment
6. Maintenance

Q4: Which is best SDLC model?

Depends on project type.

Agile is most widely used today.

Q5: Difference between Agile and Waterfall?

Agile flexible,
Waterfall sequential.

---

18. EXAMINER TRICK QUESTIONS

"Can testing happen before coding?"

Normally no.

But test planning starts early.

"Which model best for changing requirements?"

Answer:

Agile Model.

"Why Waterfall less used today?"

Because it is rigid and inflexible.

---

19. SHORT MEMORY FORMULA

Remember:

SDLC = Plan → Design → Code → Test → Deploy → Maintain

---

20. INTERVIEW BONUS QUESTION

"What happens if SDLC is not followed?"

Answer:

Software becomes unorganized, delayed, costly, and error-prone.

TOPIC SUMMARY

SDLC is foundation of software engineering.

Without SDLC:

No structured software development.

With SDLC:

Everything becomes systematic.

When ready, type:

NEXT

Then I'll continue with:

TOPIC 3: AGILE & SCRUM COMPLETE OVERVIEW

Including:

- Agile manifesto
- Scrum roles
- Sprint planning

09/04/2026, 13:54:19

- Product backlog
- Scrum master role
- Real-time examples
- Full viva questions set

NEXT

09/04/2026, 13:54:58

TOPIC 3: AGILE & SCRUM COMPLETE OVERVIEW

(Complete Theory for DevOps Viva + Interview Preparation)

---

1. INTRODUCTION TO AGILE

Agile is one of the most important software development methodologies used today.

Agile means:

Fast, Flexible, Iterative Development

Instead of building full software at once,
software is built in small small parts.

Traditional Method Problem:

Earlier in Waterfall:

- Full software built at once
- Changes difficult
- Late feedback

Example:

If customer changes requirement late → huge problem.

Agile Solution:

Agile says:

> *Build small parts quickly, get feedback early, improve continuously.*

---

2. OFFICIAL DEFINITION OF AGILE

Agile is a software development methodology based on:

- Iterative development
- Customer collaboration
- Continuous improvement

---

3. WHY AGILE IS NEEDED?

Because software requirements change often.

Example:

Customer first says:

"I need login system."

Later says:

"Add OTP login too."

Agile handles such changes easily.

---

4. AGILE MANIFESTO

Agile is based on 4 values:

---

1. Individuals and interactions over processes and tools

People matter more than tools.

---

2. Working software over documentation

Software should work, not just paperwork.

---

3. Customer collaboration over contract negotiation

Continuous customer feedback is important.

---

4. Responding to change over following a plan

Adapt changes quickly.

---

5. AGILE PRINCIPLES

There are 12 principles.

Most important:

- Early delivery
- Welcome changing requirements
- Frequent releases
- Daily team collaboration
- Continuous improvement

---

6. AGILE LIFE CYCLE

Agile cycle:

```
Requirement → Planning → Design → Development → Testing → Review →
Release
```

Repeated again and again.

---

7. WHAT IS SCRUM?

Scrum is a framework used to implement Agile.

Simple meaning:

Scrum = Practical way to do Agile

Agile = Concept
Scrum = Method

---

8. WHY SCRUM IS USED?

Because Scrum organizes Agile work into manageable short cycles.

---

9. WHAT IS SPRINT?

Sprint = Fixed time period to complete tasks.

Usually:

- 1 week
- 2 weeks
- 4 weeks

Most common:

2 weeks

Example:
In one sprint team completes:

- Login page
- Signup page

---

10. SCRUM TEAM STRUCTURE

Three major roles:

A. PRODUCT OWNER

Responsible for:

- Defining requirements
- Prioritizing features

Represents customer needs.

B. SCRUM MASTER

Responsible for:

- Managing Scrum process
- Removing obstacles

Not team boss.

Only facilitator.

C. DEVELOPMENT TEAM

Actual people who:

- Code
- Test
- Build software

---

11. SCRUM ARTIFACTS

Important viva question.

Three artifacts:

---

1. Product Backlog

Master list of all features.

Example:

- Login
- Payment system

- Dashboard

---

2. Sprint Backlog

Selected tasks for current sprint.

---

3. Increment

Completed working software output.

---

12. SCRUM EVENTS

Five important events:

---

1. Sprint Planning

Team decides:

What to do in sprint?

---

2. Daily Scrum Meeting

15-minute daily meeting.

Questions asked:

1. What did I do yesterday?
2. What today?
3. Any blockers?

---

3. Sprint Review

Show completed work.

---

4. Sprint Retrospective

Discuss improvements.

---

5. Sprint Execution

Actual working period.

---

13. DAILY STANDUP MEETING

Very common viva question.

Short daily team meeting.

Purpose:

Track progress quickly.

Time:

15 minutes max.

---

14. REAL-TIME EXAMPLE OF SCRUM

Suppose building food delivery app.

Sprint 1:

- Login page

Sprint 2:

- Cart system

Sprint 3:

- Payment gateway

Each sprint gives usable feature.

---

15. AGILE VS SCRUM

Agile	Scrum
Philosophy	Framework
Broad concept	Specific implementation

Agile	Scrum
Many methods possible	One Agile method

---

16. SCRUM VS WATERFALL

Scrum	Waterfall
Flexible	Rigid
Fast feedback	Late feedback
Incremental releases	One final release

---

17. BENEFITS OF AGILE

Faster Delivery

Small releases happen quickly.

Flexibility

Easy changes allowed.

Better Customer Satisfaction

Frequent feedback.

Lower Risk

Problems found early.

---

18. COMMON AGILE TOOLS

Popular tools:

- Jira
- Trello
- Asana

---

19. COMMON SCRUM CHALLENGES

Challenge 1:

Scope changes too often

Challenge 2:

Poor communication

Challenge 3:

Improper sprint planning

---

20. IMPORTANT VIVA QUESTIONS

Q1: What is Agile?

Agile is iterative flexible software development method.

Q2: What is Scrum?

Scrum is framework used to implement Agile.

Q3: What is Sprint?

Fixed time period to complete tasks.

Q4: Scrum roles?

1. Product Owner
2. Scrum Master
3. Development Team

Q5: What is Product Backlog?

Master list of project tasks.

Q6: Difference Agile and Scrum?

Agile = concept
Scrum = implementation framework

---

21. TRICKY EXAMINER QUESTIONS

"Is Scrum mandatory in Agile?"

No.

Scrum is one Agile framework only.

Others:

- Kanban
- XP

"Can Sprint duration change?"

Yes,
but should remain fixed within sprint cycle.

“Who controls Scrum team?”

No one controls directly.

Scrum team is self-organizing.

---

22. INTERVIEW BONUS QUESTION

“Why Agile preferred in startups?”

Because startups face changing requirements frequently.

Agile adapts quickly.

---

23. AGILE IN DEVOPS

Agile helps DevOps by:

- Faster development cycles
- Continuous delivery readiness

Agile + DevOps together = modern software speed

---

24. MEMORY SHORTCUT

Remember:

Agile = Flexible Development

Scrum = Structured Agile Execution

---

25. SUMMARY IN ONE LINE

Agile builds software fast in small cycles, Scrum manages those cycles efficiently.

When ready type:

NEXT

Then we continue with:

TOPIC 4: GIT & GITHUB COMPLETE OVERVIEW

Including:

- Version control concept
- Git architecture
- Branching
- Merge conflicts
- Git commands
- GitHub workflow
- Fork vs Clone
- Pull Requests
- Real-time examples

09/04/2026, 13:55:30

NEXT

09/04/2026, 13:55:34

TOPIC 4: GIT & GITHUB COMPLETE OVERVIEW

(Full DevOps Viva Theory + Practical Understanding)

---

1. INTRODUCTION TO VERSION CONTROL

Before understanding Git, first understand:

What is Version Control?

Version control is a system that tracks changes made to files over time.

It helps developers:

- Save history of code
- Compare versions
- Restore old versions
- Work together safely

Example Without Version Control:

Suppose 3 developers edit same project:

```
project_final.py
project_final_latest.py
project_final_latest2.py
project_final_corrected_final.py
```

Confusing and messy.

Example With Version Control:

Git keeps everything organized:

- Who changed code
- When changed
- Why changed

---

2. WHAT IS GIT?

Git is a distributed version control system.

Created by:

Linus Torvalds

Official Meaning:

Git tracks source code changes during software development.

---

3. WHY GIT IS NEEDED?

Git solves major problems:

✔ Team collaboration
✔ Backup of code history
✔ Branching support
✔ Merge support
✔ Rollback possible

---

4. WHAT IS GITHUB?

GitHub is cloud platform for hosting Git repositories.

Git = Tool

GitHub = Online platform

GitHub

Example:
Git works locally in system.
GitHub stores repository online.

---

5. DIFFERENCE BETWEEN GIT AND GITHUB

Git	GitHub
Version control tool	Cloud hosting platform
Runs locally	Runs online
Tracks changes	Stores repositories

---

6. GIT ARCHITECTURE

Git has 3 main areas:

A. WORKING DIRECTORY

Where files are edited.

B. STAGING AREA

Temporary preparation area before commit.

C. LOCAL REPOSITORY

Permanent saved history.

Flow:

```
Working Directory → Staging Area → Local Repository → GitHub Remote Repository
```

---

7. WHAT IS REPOSITORY?

Repository = Storage location for project files.

Types:

1. Local Repository
2. Remote Repository

---

8. BASIC GIT WORKFLOW

Normal process:

1. Create file
2. Add file
3. Commit changes
4. Push to GitHub

---

9. IMPORTANT GIT COMMANDS

---

git init

Creates new repository.

```
git init
```

---

git clone

Copies existing repository.

```
git clone repository_url
```

---

git status

Shows file state.

```
git status
```

---

git add

Adds files to staging area.

```
git add .
```

---

git commit

Saves snapshot permanently.

```
git commit -m "Added login page"
```

---

git push

Uploads to GitHub.

```
git push origin main
```

---

git pull

Downloads latest updates.

```
git pull origin main
```

---

10. WHAT IS COMMIT?

Commit = Saved checkpoint in project history.

Each commit has:

- Unique ID
- Message
- Timestamp

Example:
"Fixed login bug"

---

11. WHAT IS BRANCH?

Branch = Separate development line.

Used when:
Different features developed independently.

Example:

Main branch:

```
main
```

Feature branch:

```
feature-login
```

---

12. WHY BRANCHING IMPORTANT?

Allows:

✔ Safe experimentation

✔ Parallel development

✔ No damage to main code

---

13. CREATE BRANCH COMMAND

```
git branch feature1
git checkout feature1
```

Or:

```
git checkout -b feature1
```

---

14. WHAT IS MERGING?

Combining one branch into another.

Example:

feature-login → main

```
git merge feature1
```

15. WHAT IS MERGE CONFLICT?

Occurs when same file line changed differently.

Example:

Two developers edit same line.

Git gets confused.

Resolution:

Manual correction required.

---

16. WHAT IS GITHUB REMOTE REPOSITORY?

Online copy stored on GitHub servers.

Example:

Project stored in cloud.

---

17. WHAT IS FORK?

Fork = Copy another person's repository into your account.

Used in open-source projects.

---

18. FORK VS CLONE

Fork	Clone
Server copy	Local copy
GitHub account copy	Computer copy

---

19. WHAT IS PULL REQUEST (PR)?

Request to merge code into main branch.

Example:

Developer says:

"Please review my code."

---

20. GITHUB WORKFLOW

Standard workflow:

```
Clone → Branch → Edit → Commit → Push → Pull Request → Merge
```

---

21. REAL-TIME EXAMPLE

Suppose team builds shopping app:

Developer A:

Works on login page

Developer B:

Works on cart page

Both create separate branches.

Later merged into main.

---

22. WHAT IS .GITIGNORE FILE?

Used to ignore unnecessary files.

Example:

```
node_modules/
.env
temp/
```

---

23. WHAT IS HEAD IN GIT?

HEAD points to current active branch.

Usually:

```
HEAD → main
```

---

24. GIT RESET VS REVERT

---

git reset

Deletes commit history.

---

git revert

Creates reverse commit safely.

---

25. GIT ADVANTAGES

✔ Fast
✔ Reliable
✔ Distributed
✔ Secure
✔ Collaboration friendly

---

26. COMMON GIT ERRORS

Error: Merge conflict

Fix manually.

Error: Detached HEAD

Checkout proper branch.

Error: Push rejected

Need pull first.

---

27. IMPORTANT VIVA QUESTIONS

Q1: What is Git?

Distributed version control system.

Q2: What is GitHub?

Cloud hosting platform for Git repositories.

Q3: Difference between clone and fork?

Clone = local copy
Fork = account copy online

Q4: What is branch?

Separate line of development.

Q5: What is merge conflict?

Conflict when same code line modified differently.

Q6: What is commit?

Saved snapshot of changes.

---

28. TRICKY EXAMINER QUESTIONS

"Can Git work without GitHub?"

Yes.

Git works independently.

"Can two people work same branch?"

Yes, but conflicts possible.

"Is Git centralized?"

No.

Git is distributed.

---

29. INTERVIEW BONUS QUESTION

"Why Git better than older version systems?"

Because Git is:

- Faster
- Distributed
- Supports branching efficiently

---

30. GIT IN DEVOPS

Git is foundation of DevOps because:

Every CI/CD pipeline starts from Git code commit.

Example:

Git commit triggers:

→ Jenkins build

→ Docker image

→ Kubernetes deployment

---

31. MEMORY SHORTCUT

Remember:

Git = Track Changes

GitHub = Store Online

---

32. SUMMARY IN ONE LINE

Git manages code history; GitHub enables collaboration online.

When ready type:

NEXT

Then we continue with:

TOPIC 5: CI/CD COMPLETE OVERVIEW

Including:

- Continuous Integration
- Continuous Delivery
- Continuous Deployment
- Pipeline architecture
- CI/CD real-time examples

09/04/2026, 15:55:55

NEXT

- Jenkins integration
- Viva questions in depth

TOPIC 5: CI/CD COMPLETE OVERVIEW

(Continuous Integration / Continuous Delivery / Continuous Deployment – Full DevOps Theory)

---

1. INTRODUCTION TO CI/CD

CI/CD is the backbone of modern DevOps.

Whenever developers write code, software must:

- Build correctly
- Test automatically
- Deploy quickly

CI/CD makes this happen.

Full Form:

- CI = Continuous Integration
- CD = Continuous Delivery / Continuous Deployment

---

2. WHAT IS CI/CD?

CI/CD is an automated process that helps teams:

✔ Integrate code continuously
✔ Test automatically
✔ Deliver software faster
✔ Reduce manual deployment errors

---

3. WHY CI/CD IS NEEDED?

Before CI/CD:

Problems:

- Manual testing slow

- Deployment risky
- Bugs found late
- Releases delayed

Example:

Without CI/CD:

Deployment may take days.

With CI/CD:

Deployment can happen in minutes.

---

4. WHAT IS CONTINUOUS INTEGRATION (CI)?

Continuous Integration means:

Developers merge code regularly into shared repository.

Every code change triggers:

1. Build
2. Test
3. Validation

Example:

Developer commits code to Git.

Immediately:

- Code compiled
- Tests run automatically

Goal of CI:

Detect bugs early.

---

5. WHAT IS CONTINUOUS DELIVERY?

Continuous Delivery means:

Software is always ready for deployment.

After testing:

System prepares release automatically.

Human approval may still be needed.

---

6. WHAT IS CONTINUOUS DEPLOYMENT?

Continuous Deployment means:

Software is automatically deployed without manual approval.

Example:

Code passes tests → instantly live.

---

7. DIFFERENCE BETWEEN CI, CD, AND DEPLOYMENT

Term	Meaning
CI	Code integration + build + test
Continuous Delivery	Ready for release
Continuous Deployment	Auto release to production

---

8. CI/CD PIPELINE

Pipeline = automated sequence of stages.

Typical pipeline:

```
Code Commit → Build → Test → Package → Deploy → Monitor
```

---

9. STAGES OF CI/CD PIPELINE

Stage 1: SOURCE CODE

Developer pushes code to Git.

Example:

GitHub repository stores code.

GitHub

Stage 2: BUILD

Application compiled.

Example:

Java project uses Maven.

Stage 3: TEST

Automated tests executed.

Tools:

- Selenium
- JUnit

Stage 4: PACKAGE

Application packaged.

Example:

Docker image created.

Docker

Stage 5: DEPLOY

Application deployed.

Example:

Kubernetes cluster deployment.

Kubernetes

Stage 6: MONITOR

Application health monitored.

Tools:

- Prometheus
- Grafana

---

10. REAL-TIME EXAMPLE OF CI/CD

Suppose company develops shopping app.

Developer updates payment feature.

Flow:

1. Code pushed to GitHub
2. Jenkins detects change
3. Build starts
4. Tests run
5. Docker image built
6. Kubernetes deploys app

---

11. WHAT IS CI SERVER?

CI server automates build/test tasks.

Popular CI servers:

- Jenkins
- GitLab CI
- CircleCI

---

12. WHY JENKINS USED IN CI/CD?

Jenkins automates:

✔ Build

✔ Test

✔ Deployment

It is central automation engine.

---

13. BENEFITS OF CI/CD

Faster Releases

Deployment happens quickly.

Early Bug Detection

Bugs caught before production.

Less Human Error

Automation reduces mistakes.

Better Software Quality

Continuous testing improves quality.

---

14. CI/CD IN DEVOPS

CI/CD is heart of DevOps because:

DevOps requires automation.

Without CI/CD:

DevOps incomplete.

---

15. COMMON CI/CD TOOLS

Category	Tool
Version Control	Git

Category	Tool
CI Server	Jenkins
Container	Docker
Orchestration	Kubernetes
Testing	Selenium

---

16. PIPELINE TYPES

Build Pipeline

Compiles software.

Release Pipeline

Deploys software.

End-to-End Pipeline

Full automation chain.

---

17. MANUAL VS AUTOMATED DEPLOYMENT

Manual	Automated
Slow	Fast
Risky	Reliable
Human errors	Fewer errors

---

18. COMMON CI/CD CHALLENGES

Challenge 1:

Failed builds

Challenge 2:

Broken tests

Challenge 3:

Pipeline misconfiguration

Challenge 4:

Deployment failures

---

19. FAILURE EXAMPLE

If one test fails:

Pipeline stops automatically.

Reason:

Prevent broken code deployment.

---

20. BLUE-GREEN DEPLOYMENT

Advanced deployment strategy.

Two environments:

- Blue = current live
- Green = new version

Switch traffic safely.

---

21. ROLLBACK IN CI/CD

If deployment fails:

Previous stable version restored.

Important for production safety.

---

22. IMPORTANT VIVA QUESTIONS

Q1: What is CI?

Automatic code integration and testing.

Q2: What is Continuous Delivery?

Software always ready for release.

Q3: What is Continuous Deployment?

Automatic production deployment.

Q4: Difference Delivery vs Deployment?

Delivery needs approval.

Deployment fully automatic.

Q5: What is CI/CD pipeline?

Automated software build-test-deploy flow.

---

23. TRICKY EXAMINER QUESTIONS

“Can CI exist without CD?”

Yes.

CI can work independently.

“Can CD happen without CI?”

No.

CI is foundation for CD.

“Why pipeline stops after failed test?”

To avoid bad code reaching production.

---

24. INTERVIEW BONUS QUESTION

“What happens when developer pushes code?”

Answer:

1. Trigger pipeline
2. Build starts
3. Tests run
4. Package created
5. Deploy if successful

---

25. REAL COMPANY USAGE

Companies using CI/CD:

- Amazon
- Netflix
- Google

They deploy many times daily.

---

26. MEMORY SHORTCUT

Remember:

CI = Integrate Code Frequently

CD = Deliver/Deploy Automatically

---

27. SUMMARY IN ONE LINE

CI/CD automates software integration, testing, and deployment for faster reliable releases.

When ready type:

NEXT

Then we continue with:

TOPIC 6: JENKINS COMPLETE OVERVIEW

Including:

- Jenkins architecture
- Master-agent
- Pipelines
- Jenkinsfile
- Plugins
- Real-time CI/CD integration
- Full viva questions

09/04/2026, 13:56:19

NEXT

09/04/2026, 13:57:00

TOPIC 6: JENKINS COMPLETE OVERVIEW

(Full DevOps Viva Theory + Real-Time Practical Understanding)

---

1. INTRODUCTION TO JENKINS

Jenkins is one of the most important DevOps tools.

It is used to automate:

- Build process
- Testing
- Deployment

Officially:

Jenkins is an open-source automation server.

Jenkins

---

2. WHY JENKINS IS NEEDED?

Before Jenkins:

Problems:

- Manual build process
- Slow deployment
- Repetitive tasks
- Human errors

Jenkins solves by:
✔ Automation
✔ Speed
✔ Reliability

---

3. MAIN PURPOSE OF JENKINS

Jenkins automates CI/CD pipeline.

Example:

When developer pushes code:

1. Jenkins detects change
2. Builds project
3. Runs tests

4. Deploys application

---

4. HISTORY OF JENKINS

Originally called:

Hudson

Later renamed Jenkins.

Created by:
Kohsuke Kawaguchi

---

5. WHAT CAN JENKINS DO?

Jenkins can automate:

- Code build
- Unit testing
- Integration testing
- Docker builds
- Kubernetes deployment

---

6. JENKINS ARCHITECTURE

Very important viva topic.

Jenkins uses:

Master-Agent Architecture

A. MASTER NODE

Controls:

- Scheduling jobs
- Managing pipelines

- Monitoring builds

B. AGENT NODE

Executes tasks assigned by master.

Example:

Build project on remote machine.

Architecture:

```
Developer → GitHub → Jenkins Master → Jenkins Agent → Build/Test/Deploy
```

GitHub

---

7. WHAT IS A JOB IN JENKINS?

Job = Task performed by Jenkins.

Examples:

- Build Java app
- Run tests
- Deploy Docker container

---

8. TYPES OF JENKINS JOBS

Important types:

1. Freestyle Project
2. Pipeline Project
3. Multi-configuration Project

---

9. WHAT IS JENKINS PIPELINE?

Pipeline = Series of automated steps.

Example stages:

```
Build → Test → Deploy
```

---

10. TYPES OF PIPELINES

Declarative Pipeline

Simple syntax.

Scripted Pipeline

Advanced Groovy-based scripting.

---

11. WHAT IS JENKINSFILE?

Jenkinsfile = Text file defining pipeline steps.

Stored inside project repository.

Example:

```
pipeline {
 agent any
 stages {
   stage('Build') {
     steps {
       echo 'Building project'
     }
   }
 }
}
```

---

12. JENKINS PIPELINE STAGES

Typical stages:

Stage 1: Source Code Fetch

Pull code from GitHub.

Stage 2: Build

Compile code.

Stage 3: Test

Run automated tests.

Stage 4: Deploy

Deploy application.

---

13. WHAT ARE PLUGINS IN JENKINS?

Plugins extend Jenkins features.

Example plugins:

- Git plugin
- Docker plugin
- Kubernetes plugin

Why plugins important?

Because Jenkins core is minimal.

Plugins make it powerful.

---

14. COMMONLY USED JENKINS PLUGINS

Popular plugins:

1. Git Plugin
2. Pipeline Plugin
3. Docker Plugin
4. Blue Ocean Plugin

---

15. HOW JENKINS WORKS IN REAL TIME

Example workflow:

Developer pushes code → GitHub

Then Jenkins:

1. Detects commit
2. Pulls code
3. Builds app
4. Runs tests
5. Creates Docker image
6. Deploys app

---

16. JENKINS + GIT INTEGRATION

Git triggers Jenkins using:

Webhooks

Whenever code pushed:
Automatic build starts.

---

17. JENKINS + DOCKER INTEGRATION

Jenkins can:

- Build Docker image
- Push to Docker Hub

Docker

---

18. JENKINS + KUBERNETES INTEGRATION

Jenkins can deploy containers into:

Kubernetes

Example:

Auto deployment after successful build.

---

19. FREESTYLE VS PIPELINE JOB

Freestyle	Pipeline
Simple	Advanced
GUI based	Code based
Less flexible	More flexible

---

20. WHAT IS BUILD TRIGGER?

Trigger starts job automatically.

Examples:

- Code commit
- Scheduled time
- Manual click

---

21. COMMON BUILD STATUS IN JENKINS

SUCCESS

Build passed.

FAILURE

Build failed.

UNSTABLE

Tests failed partially.

ABORTED

Stopped manually.

---

22. WHAT IS BLUE OCEAN?

Modern Jenkins UI plugin.

Provides better pipeline visualization.

---

23. SECURITY IN JENKINS

Security includes:

- User authentication
- Role permissions
- Credential management

---

24. COMMON JENKINS ERRORS

Plugin failure

Version mismatch.

Build failed

Code issue.

Agent offline

Connection problem.

---

25. ADVANTAGES OF JENKINS

✔ Open source
✔ Huge plugin ecosystem
✔ Easy CI/CD automation
✔ Scalable

---

26. DISADVANTAGES OF JENKINS

✘ Plugin dependency issues
✘ Complex setup sometimes
✘ Maintenance required

---

27. IMPORTANT VIVA QUESTIONS

Q1: What is Jenkins?

Automation server for CI/CD.

Q2: Why Jenkins used?

To automate build, test, deployment.

Q3: What is Jenkinsfile?

Pipeline definition file.

Q4: What is Jenkins pipeline?

Automated sequence of stages.

Q5: What is Master-Agent architecture?

Master controls jobs,
Agent executes jobs.

Q6: What are plugins?

Extensions adding features.

---

28. TRICKY EXAMINER QUESTIONS

"Can Jenkins work without plugins?"

Yes, but limited functionality.

"Is Jenkins only for Java?"

No.

Supports many languages.

"Can Jenkins run on Windows?"

Yes.

Linux, Windows, macOS supported.

---

29. INTERVIEW BONUS QUESTION

“Why Jenkins popular in DevOps?”

Because:

- Flexible
- Free
- Powerful automation support

---

30. REAL COMPANY USAGE

Companies using Jenkins:

- Netflix
- Amazon
- Facebook

---

31. MEMORY SHORTCUT

Remember:

Jenkins = Automation Brain of CI/CD

---

32. SUMMARY IN ONE LINE

Jenkins automates software build, testing, and deployment in DevOps pipelines.

When ready type:

NEXT

Then we continue with:

TOPIC 7: DOCKER COMPLETE OVERVIEW

Including:

- Containers concept
- Docker architecture
- Dockerfile
- Images vs Containers
- Volumes
- Networking
- Real-time deployment examples
- Full viva questions

09/04/2026, 13:57:36

NEXT

09/04/2026, 13:57:36

TOPIC 7: DOCKER COMPLETE OVERVIEW

(Full DevOps Viva Theory + Practical Understanding)

---

1. INTRODUCTION TO DOCKER

Docker is one of the most essential DevOps tools.

It is used for:

Containerization

Docker allows developers to package application + dependencies together.

Officially:

Docker is a containerization platform.

Docker

---

2. WHY DOCKER IS NEEDED?

Before Docker:

Problem:

Application works on one machine but fails on another.

Example:

Developer says:

> *"It works on my laptop."*

Server says:

> *"It doesn't work here."*

Docker solves this.

---

3. WHAT IS CONTAINERIZATION?

Containerization means:

Packaging:

- Application
- Libraries
- Dependencies
- Runtime

Into one portable unit called container.

---

4. WHAT IS A CONTAINER?

Container is lightweight isolated software package.

Contains:
✔ Code
✔ Runtime
✔ Libraries
✔ Dependencies

Example:

A Node.js app inside container includes:

- Node runtime
- App files
- Required packages

5. WHAT IS DOCKER IMAGE?

Image = Blueprint/template of container.

Example:

Image creates multiple containers.

Analogy:

Image = Class

Container = Object

---

6. IMAGE VS CONTAINER

Image	Container
Template	Running instance
Static	Active process

---

7. DOCKER VS VIRTUAL MACHINE

Very important viva question.

Docker Container	Virtual Machine
Lightweight	Heavy
Shares OS kernel	Full OS required
Fast startup	Slow startup

Why Docker faster?

Because containers do not load full operating system.

---

8. DOCKER ARCHITECTURE

Docker architecture has 3 main parts:

A. Docker Client

Where commands are given.

Example:

```
docker run nginx
```

B. Docker Daemon

Processes Docker commands.

C. Docker Registry

Stores Docker images.

Example:

Docker Hub

Architecture:

```
Docker Client → Docker Daemon → Docker Registry
```

---

9. WHAT IS DOCKER HUB?

Cloud repository for Docker images.

Example:

Download official images like:

- Ubuntu
- Nginx

- MySQL

---

10. IMPORTANT DOCKER COMPONENTS

---

1. Images

Blueprint files

---

2. Containers

Running applications

---

3. Dockerfile

Instructions to create image

---

4. Volumes

Persistent storage

---

5. Networks

Communication between containers

---

11. WHAT IS DOCKERFILE?

Dockerfile is text file containing instructions to build image.

Example:

```
FROM node:18
WORKDIR /app
COPY . .
```

```
RUN npm install
CMD ["npm","start"]
```

Explanation:

FROM → base image

COPY → copy files

RUN → execute command

CMD → startup command

---

12. BASIC DOCKER WORKFLOW

Flow:

```
Dockerfile → Build Image → Run Container
```

---

13. IMPORTANT DOCKER COMMANDS

---

docker pull

Download image:

```
docker pull nginx
```

---

docker images

List images:

```
docker images
```

---

docker run

Run container:

```
docker run nginx
```

---

docker ps

List running containers:

```
docker ps
```

---

docker stop

Stop container:

```
docker stop container_id
```

---

docker rm

Remove container:

```
docker rm container_id
```

---

docker rmi

Delete image:

```
docker rmi image_id
```

---

14. WHAT IS DOCKER VOLUME?

Volumes store persistent data.

Without volume:

Data lost when container deleted.

Example:

Database storage needs volume.

---

15. WHAT IS DOCKER NETWORK?

Allows containers communicate.

Example:

Web app container talks to database container.

---

16. TYPES OF DOCKER NETWORKS

1. Bridge
2. Host
3. None
4. Overlay

---

17. WHAT IS PORT MAPPING?

Maps container port to host machine.

Example:

```
docker run -p 8080:80 nginx
```

Meaning:

Host port 8080 → Container port 80

---

18. REAL-TIME EXAMPLE

Suppose deploying web app:

Step 1:

Create Dockerfile

Step 2:

Build image

```
docker build -t myapp .
```

Step 3:

Run container

```
docker run -p 3000:3000 myapp
```

---

19. WHAT IS DOCKER COMPOSE?

Used for multi-container applications.

Example:

App + Database together.

Example compose file:

```
version: '3'
services:
  web:
    image: nginx
  db:
    image: mysql
```

---

20. DOCKER IN DEVOPS

Docker helps DevOps by:

✔ Consistent deployment

✔ Easy scaling
✔ Fast delivery

---

21. DOCKER + JENKINS INTEGRATION

Jenkins can:

- Build Docker image
- Push to registry

Jenkins

---

22. DOCKER + KUBERNETES

Docker creates containers.

Kubernetes manages them.

Kubernetes

---

23. ADVANTAGES OF DOCKER

✔ Fast deployment
✔ Lightweight
✔ Portable
✔ Scalable

---

24. DISADVANTAGES OF DOCKER

✘ Security concerns
✘ Complex networking sometimes

---

25. COMMON DOCKER ERRORS

Error: Container exits immediately

Reason:
Main process ended.

Error: Port already in use

Reason:
Another app using same port.

Error: Image not found

Reason:
Wrong image name.

---

26. IMPORTANT VIVA QUESTIONS

Q1: What is Docker?

Containerization platform.

Q2: What is container?

Lightweight isolated package.

Q3: Difference image vs container?

Image = template
Container = running instance

Q4: What is Dockerfile?

Instruction file for image creation.

Q5: Difference Docker vs VM?

Docker lighter and faster.

---

27. TRICKY EXAMINER QUESTIONS

"Can one image create many containers?"

Yes.

One image → multiple containers.

"Can Docker run without internet?"

Yes, if images already available locally.

"Does Docker replace virtual machines?"

Not fully.

Both have different uses.

---

28. INTERVIEW BONUS QUESTION

"Why Docker became popular?"

Because it solves environment inconsistency problem.

---

29. REAL COMPANY USAGE

Companies using Docker:

- Spotify
- Netflix
- PayPal

---

30. MEMORY SHORTCUT

Remember:

Docker = Build Once, Run Anywhere

---

31. SUMMARY IN ONE LINE

Docker packages applications into portable containers for consistent deployment everywhere.

When ready type:

NEXT

Then we continue with:

TOPIC 8: KUBERNETES COMPLETE OVERVIEW

Including:

- Pod concept
- Deployments
- Services
- ReplicaSets
- Cluster architecture
- Master & worker nodes
- Minikube

09/04/2026, 13:57:38

- Full viva questions

NEXT

09/04/2026, 13:58:18

TOPIC 8: KUBERNETES COMPLETE OVERVIEW

(Full DevOps Viva Theory + Real-Time Practical Understanding)

---

1. INTRODUCTION TO KUBERNETES

Kubernetes is one of the most powerful DevOps tools.

It is used for:

Container Orchestration

Officially:

Kubernetes is an open-source container orchestration platform.

Kubernetes

Short Name:

Kubernetes is also called:

K8s

(K + 8 letters + s)

---

2. WHY KUBERNETES IS NEEDED?

Docker creates containers well.

But problem arises when:

- Hundreds of containers run together
- Need scaling
- Need load balancing
- Need self-healing

Docker alone cannot manage large-scale container systems efficiently.

Kubernetes solves this.

---

3. REAL-WORLD PROBLEM WITHOUT KUBERNETES

Suppose company runs:

500 Docker containers.

Without Kubernetes:

✘ Hard to manage manually

✘ Difficult scaling

✘ Failure recovery slow

With Kubernetes:

✔ Automatic management

✔ Self-healing

✔ Auto scaling

---

4. WHAT DOES KUBERNETES DO?

Kubernetes manages:

- Deployment
- Scaling
- Networking
- Load balancing
- Recovery

---

5. SIMPLE DEFINITION

Docker creates containers

Kubernetes manages containers

---

6. KUBERNETES ARCHITECTURE

Very important viva topic.

Cluster has:

1. Master Node (Control Plane)

2. Worker Nodes

Architecture:

```
Master Node
   ↓
Worker Node 1 → Pods
Worker Node 2 → Pods
Worker Node 3 → Pods
```

---

7. MASTER NODE COMPONENTS

Master node controls cluster.

Main components:

A. API Server

Entry point for commands.

Example:

kubectl communicates here.

B. Scheduler

Assigns pods to worker nodes.

C. Controller Manager

Maintains desired state.

D. etcd

Stores cluster data.

8. WORKER NODE COMPONENTS

Worker node runs containers.

Includes:

A. Kubelet

Agent running on each node.

B. Kube Proxy

Handles networking.

C. Container Runtime

Runs containers.

Example:

Docker / containerd

---

9. WHAT IS A CLUSTER?

Cluster = Group of machines running Kubernetes.

---

10. WHAT IS POD?

Most important viva question.

Pod = Smallest deployable unit in Kubernetes

Pod contains:

- One or more containers

Example:

1 pod may contain:

- Web container
- Logging container

---

11. WHY POD NEEDED?

Because Kubernetes does not deploy containers directly.

It deploys pods.

---

12. WHAT IS DEPLOYMENT?

Deployment manages pod creation and updates.

Example:

Need 5 pods running always.

Deployment ensures this.

---

13. WHAT IS REPLICASET?

ReplicaSet ensures fixed number of pod copies.

Example:

Desired = 3 pods

If one crashes:

Automatically recreated.

---

14. WHAT IS SERVICE?

Service exposes pods to network.

Without service:

Pods inaccessible externally.

---

15. TYPES OF SERVICES

Important viva question.

A. ClusterIP

Internal communication only.

B. NodePort

External access via node port.

C. LoadBalancer

External cloud load balancing.

---

16. WHAT IS NAMESPACE?

Namespace divides cluster resources logically.

Example:

- dev namespace
- prod namespace

---

17. WHAT IS CONFIGMAP?

Stores configuration data.

Example:

Database URL settings.

---

18. WHAT IS SECRET?

Stores sensitive data securely.

Example:

Passwords

API keys

---

19. WHAT IS INGRESS?

Ingress manages external HTTP routing.

Example:

Route traffic to correct service.

---

20. KUBERNETES WORKFLOW

Flow:

```
Developer → YAML File → API Server → Scheduler → Worker Node → Pod Runs
```

---

21. YAML IN KUBERNETES

Configurations written in YAML files.

Example deployment:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
```

---

22. IMPORTANT KUBECTL COMMANDS

Get pods

```
kubectl get pods
```

Get services

```
kubectl get services
```

Apply config

```
kubectl apply -f deployment.yaml
```

Delete pod

```
kubectl delete pod podname
```

Describe pod

```
kubectl describe pod podname
```

---

23. WHAT IS SELF-HEALING?

If pod crashes:

Kubernetes automatically recreates it.

---

24. WHAT IS AUTO SCALING?

Kubernetes increases/decreases pods automatically.

Example:

High traffic → add more pods

---

25. WHAT IS LOAD BALANCING?

Traffic distributed among pods evenly.

Example:

100 users spread across multiple pods.

---

26. WHAT IS ROLLING UPDATE?

Updates app without downtime.

Old pods replaced gradually.

---

27. WHAT IS MINIKUBE?

Minikube creates local Kubernetes cluster on laptop.

Used for learning/testing.

Minikube

---

28. DOCKER VS KUBERNETES

Docker	Kubernetes
Creates containers	Manages containers
Single host focus	Multi-node orchestration

---

29. KUBERNETES IN DEVOPS

Kubernetes helps DevOps by:

✔️ Automated deployment

✔ Scalability
✔ Reliability

---

30. COMMON KUBERNETES ERRORS

CrashLoopBackOff

Pod repeatedly crashing.

ImagePullBackOff

Cannot download image.

Pending Pod

Insufficient resources.

---

31. IMPORTANT VIVA QUESTIONS

Q1: What is Kubernetes?

Container orchestration platform.

Q2: What is Pod?

Smallest deployable unit.

Q3: What is Deployment?

Manages pod lifecycle.

Q4: What is ReplicaSet?

Maintains desired pod count.

Q5: What is Service?

Exposes pods to network.

Q6: Difference Docker vs Kubernetes?

Docker creates containers,
Kubernetes manages them.

---

32. TRICKY EXAMINER QUESTIONS

"Can pod have multiple containers?"

Yes.

"Can Kubernetes work without Docker?"

Yes.

Supports other runtimes too.

"Why pod not container directly?"

Pod adds networking and management abstraction.

---

33. INTERVIEW BONUS QUESTION

"Why Kubernetes popular?"

Because it automates large-scale container management.

---

34. REAL COMPANY USAGE

Companies using Kubernetes:

- Google
- Spotify
- Airbnb

---

35. MEMORY SHORTCUT

Remember:

Docker builds containers

Kubernetes controls them

---

36. SUMMARY IN ONE LINE

Kubernetes automates deployment, scaling, and management of containerized applications.

When ready type:

NEXT

Then we continue with:

TOPIC 9: SELENIUM COMPLETE OVERVIEW

Including:

09/04/2026, 13:58:20

- Automation testing basics
- Selenium architecture
- WebDriver

NEXT

09/04/2026, 13:58:59

TOPIC 9: SELENIUM COMPLETE OVERVIEW

(Full DevOps Viva Theory + Testing Automation Understanding)

---

1. INTRODUCTION TO SELENIUM

Selenium is one of the most widely used tools in software testing.

It is used for:

Web Application Automation Testing

Officially:

Selenium is an open-source web automation testing framework.

Selenium

---

2. WHY SELENIUM IS NEEDED?

Before Selenium:

Testing was manual:

- Slow
- Repetitive
- Error-prone

Example:
If website has 100 pages,
manual testing takes hours.

Selenium automates this.

---

3. WHAT IS AUTOMATION TESTING?

Automation testing means:

Using software tools to test applications automatically instead of manually.

Example:

Automatically checking:

- Login page
- Buttons
- Forms
- Links

---

4. MANUAL TESTING VS AUTOMATION TESTING

Manual Testing	Automation Testing
Human performs test	Tool performs test
Slow	Fast
Repetitive tiring	Reusable scripts

---

5. WHAT SELENIUM CAN TEST?

Selenium tests:

✔ Web applications

✔ Browser functionality

✔ UI behavior

Selenium cannot test:

✘ Desktop apps directly

✘ Mobile apps directly (without Appium)

---

6. SELENIUM COMPONENTS

Very important viva question.

Selenium has 4 major components:

A. Selenium IDE

Record and playback tool.

Beginner friendly.

B. Selenium WebDriver

Main automation engine.

Most important component.

C. Selenium Grid

Runs tests on multiple systems simultaneously.

D. Selenium RC (Old)

Deprecated now.

---

7. WHAT IS SELENIUM WEBDRIVER?

WebDriver controls browser automatically.

Example:

Open browser → click button → enter text

Example code:

```
from selenium import webdriver
driver = webdriver.Chrome()
driver.get("https://google.com")
```

---

8. SUPPORTED BROWSERS

Selenium supports:

- Google Chrome
- Mozilla Firefox

- Microsoft Edge
- Safari

---

9. HOW SELENIUM WORKS

Flow:

```
Test Script → WebDriver → Browser Driver → Browser
```

Example:

Python script → ChromeDriver → Chrome browser

---

10. WHAT IS BROWSER DRIVER?

Browser driver acts as bridge.

Examples:

- ChromeDriver
- GeckoDriver

---

11. WHAT IS SELENIUM GRID?

Grid allows:

Multiple tests on:

- Different browsers
- Different machines

Example:

Same test on Chrome + Firefox together.

---

12. HUB AND NODE IN GRID

Hub:

Central controller.

Node:

Machines executing tests.

Architecture:

```
Hub → Node1 Chrome
    → Node2 Firefox
    → Node3 Edge
```

---

13. PROGRAMMING LANGUAGES SUPPORTED

Selenium supports:

- Java
- Python
- C#
- JavaScript
- Ruby

---

14. COMMON SELENIUM ACTIONS

Selenium can:

✔ Open page
✔ Click buttons
✔ Enter text
✔ Select dropdown
✔ Submit forms

Example:

```
driver.find_element("id","username").send_keys("admin")
```

---

15. LOCATORS IN SELENIUM

Important viva topic.

Locators identify web elements.

Types:

1. ID
2. Name
3. XPath
4. CSS Selector
5. Class Name

---

16. WHAT IS XPATH?

XPath locates HTML elements.

Example:

```
//*[@id='login']
```

---

17. TEST CASE IN SELENIUM

Test case = sequence of steps verifying function.

Example:

Login Test Case:

1. Open login page
2. Enter username
3. Enter password
4. Click login

18. ASSERTIONS IN SELENIUM

Used to verify expected results.

Example:

Check page title.

19. SELENIUM IN CI/CD PIPELINE

Important DevOps integration.

Flow:

```
Code Commit → Jenkins Build → Selenium Test → Deploy
```

Jenkins

20. SELENIUM + JENKINS

Jenkins can automatically:

- Trigger Selenium tests
- Generate reports

21. SELENIUM + DOCKER

Selenium Grid can run inside Docker containers.

Docker

22. ADVANTAGES OF SELENIUM

✔ Open source
✔ Multi-browser support
✔ Multi-language support
✔ Reusable scripts

23. DISADVANTAGES OF SELENIUM

✘ Web only

✘ No built-in reporting

✘ Requires coding knowledge

---

24. COMMON SELENIUM ERRORS

NoSuchElementException

Element not found.

TimeoutException

Page load delay.

StaleElementReferenceException

Element changed after loading.

---

25. REAL-TIME EXAMPLE

Suppose testing e-commerce site:

Automated script checks:

- Login works
- Cart adds product
- Payment page opens

All without manual work.

---

26. TESTING TYPES USING SELENIUM

Selenium mainly supports:

- Functional testing
- Regression testing
- Smoke testing

---

27. WHAT IS REGRESSION TESTING?

Checking old features still work after updates.

---

28. WHAT IS SMOKE TESTING?

Basic check whether system works at all.

Example:

Does login page load?

---

29. IMPORTANT VIVA QUESTIONS

Q1: What is Selenium?

Automation testing framework for web apps.

Q2: What is WebDriver?

Browser automation engine.

Q3: What is Selenium Grid?

Runs parallel tests on multiple systems.

Q4: What are locators?

Methods to identify webpage elements.

Q5: Difference Selenium IDE vs WebDriver?

IDE = record/playback

WebDriver = coding-based automation

---

30. TRICKY EXAMINER QUESTIONS

"Can Selenium test desktop apps?"

No, mainly web apps.

"Does Selenium support mobile testing?"

Not directly; use Appium.

"Is Selenium only for testers?"

No, developers also use it.

---

31. INTERVIEW BONUS QUESTION

"Why Selenium popular?"

Because:

- Free
- Flexible
- Supports many browsers

---

32. REAL COMPANY USAGE

Companies using Selenium:

- Amazon

- Flipkart
- Netflix

---

33. MEMORY SHORTCUT

Remember:

Selenium = Automatic Browser Tester

---

34. SUMMARY IN ONE LINE

Selenium automates browser testing to ensure web applications work correctly.

When ready type:

NEXT

Then we continue with:

TOPIC 10: LINUX FOR DEVOPS COMPLETE OVERVIEW

Including:

- Linux basics
- File system
- Important commands
- chmod, grep, awk, sed
- Process management
- Shell scripting
- Full viva questions

09/04/2026, 13:59:00

NEXT

09/04/2026, 13:59:43

TOPIC 10: LINUX FOR DEVOPS COMPLETE OVERVIEW

(Full DevOps Viva Theory + Practical Command Understanding)

---

1. INTRODUCTION TO LINUX

Linux is one of the most important foundations of DevOps.

Most DevOps tools run mainly on Linux:

- Docker
- Kubernetes
- Jenkins

What is Linux?

Linux is an open-source operating system.

It manages:

- Hardware
- Software
- Processes
- Memory

---

2. WHY LINUX IS IMPORTANT IN DEVOPS?

Because:

✔ Most servers use Linux
✔ Stable and secure
✔ Free and open-source
✔ Powerful command-line control

Real Example:

Cloud servers in:

- Amazon Web Services
- Google Cloud

mostly run Linux systems.

---

3. WHAT IS KERNEL?

Kernel is core part of Linux OS.

It connects:

Hardware ↔ Software

---

4. LINUX ARCHITECTURE

Linux architecture:

```
Applications
Shell
Kernel
Hardware
```

Explanation:

Applications → user programs
Shell → command interpreter
Kernel → OS core
Hardware → physical machine

---

5. WHAT IS SHELL?

Shell accepts commands from user.

Example:

```
ls
pwd
mkdir
```

Popular shells:

- Bash
- Zsh

---

6. LINUX FILE SYSTEM

Linux stores everything as files.

Structure starts from:

```
/
```

This is called:

Root Directory

---

7. IMPORTANT DIRECTORIES

/home

User files

/etc

Configuration files

/bin

Basic commands

/var

Logs and variable data

/tmp

Temporary files

---

8. BASIC LINUX COMMANDS

Very important viva section.

---

pwd

Print current directory

```
pwd
```

---

ls

List files

```
ls
```

---

cd

Change directory

```
cd foldername
```

---

mkdir

Create folder

```
mkdir project
```

---

rm

Delete files

```
rm file.txt
```

---

cp

Copy files

```
cp a.txt b.txt
```

---

mv

Move/rename file

```
mv old.txt new.txt
```

---

9. FILE PERMISSIONS IN LINUX

Each file has permissions:

```
r = read
w = write
x = execute
```

Example:

```
-rwxr-xr--
```

---

10. WHAT IS chmod?

Changes file permissions.

Example:

```
chmod 755 script.sh
```

Meaning of 755:

Owner: rwx

Group: r-x

Others: r-x

---

11. WHAT IS chown?

Changes file ownership.

Example:

```
chown user file.txt
```

---

12. PROCESS MANAGEMENT

Linux runs programs as processes.

---

ps

Shows running processes

```
ps
```

---

top

Live process monitor

```
top
```

---

kill

Terminate process

```
kill PID
```

---

13. WHAT IS grep?

Searches text patterns.

Example:

```
grep "error" log.txt
```

---

14. WHAT IS sed?

Stream editor for modifying text.

Example:

```
sed 's/old/new/g' file.txt
```

---

15. WHAT IS awk?

Text processing tool.

Example:

```
awk '{print $1}' file.txt
```

---

16. WHAT IS sort?

Sort lines alphabetically.

```
sort names.txt
```

---

17. WHAT IS cat?

Display file contents.

```
cat file.txt
```

---

18. WHAT IS head AND tail?

---

head:

First lines of file

```
head file.txt
```

---

tail:

Last lines

```
tail file.txt
```

---

19. REDIRECTION OPERATORS

>

Overwrite output

```
ls > files.txt
```

>>

Append output

```
echo hello >> file.txt
```

---

20. PIPE IN LINUX

Pipe symbol:

```
|
```

Pass output from one command to another.

Example:

```
ls | grep txt
```

---

21. WHAT IS SSH?

SSH = Secure remote login.

Example:

```
ssh user@server_ip
```

---

22. WHAT IS SCP?

Secure copy between systems.

Example:

```
scp file.txt user@host:/home/
```

---

23. PACKAGE MANAGEMENT

Install software using package managers.

Ubuntu:

```
sudo apt install nginx
```

---

24. WHAT IS SHELL SCRIPTING?

Writing commands in file to automate tasks.

Example:

```
#!/bin/bash
echo Hello World
```

---

25. WHY SHELL SCRIPTING IMPORTANT IN DEVOPS?

Because DevOps needs automation.

Used for:

✔ Deployment scripts
✔ Backup automation
✔ Monitoring tasks

---

26. LINUX IN DEVOPS TOOLS

All major DevOps tools depend heavily on Linux:

- Jenkins agents
- Docker containers
- Kubernetes nodes

---

27. COMMON LINUX ERRORS

Permission denied

Fix:

Use chmod

Command not found

Cause:

Package missing

No such file

Wrong path given

---

28. IMPORTANT VIVA QUESTIONS

Q1: What is Linux?

Open-source operating system.

Q2: What is shell?

Command interpreter.

Q3: What is chmod?

Changes file permissions.

Q4: Difference grep and awk?

grep searches text,
awk processes structured text.

Q5: What is root directory?

Top-level directory (/)

---

29. TRICKY EXAMINER QUESTIONS

"Is Linux same as Unix?"

No.

Linux is Unix-like, not exact Unix.

“Why Linux preferred in servers?”

Stable, secure, lightweight.

“Can Linux run without GUI?”

Yes.

Command line only possible.

---

30. INTERVIEW BONUS QUESTION

“Why Linux essential for DevOps engineer?”

Because most DevOps infrastructure runs on Linux.

---

31. REAL COMPANY USAGE

Linux widely used by:

- Google
- Amazon
- Meta

---

32. MEMORY SHORTCUT

Remember:

Linux = Backbone of DevOps Infrastructure

---

33. SUMMARY IN ONE LINE

Linux provides the stable operating system foundation for running DevOps tools and servers.

When ready type:

NEXT

Then we continue with:

TOPIC 11: CLOUD COMPUTING FOR DEVOPS COMPLETE OVERVIEW

Including:

- Cloud basics
- IaaS / PaaS / SaaS
- AWS basics
- Azure basics
- GCP basics
- Cloud in DevOps
- Full viva questions

09/04/2026, 13:59:45

NEXT

09/04/2026, 14:00:36

TOPIC 11: CLOUD COMPUTING FOR DEVOPS COMPLETE OVERVIEW

(Full DevOps Viva Theory + Real-Time Cloud Understanding)

---

1. INTRODUCTION TO CLOUD COMPUTING

Cloud computing is one of the most important technologies in DevOps.

Modern applications are hosted in the cloud instead of local servers.

Example:
Apps like:

- Netflix
- Spotify
  run on cloud infrastructure.

Definition:

Cloud computing means:

Delivering computing services over the internet.

These services include:

- Servers
- Storage
- Databases
- Networking
- Software

---

2. WHY CLOUD IS NEEDED?

Before cloud:

Problems:

✘ Expensive hardware
✘ Physical server maintenance
✘ Limited scalability

Cloud solves:

✔ Pay only for usage
✔ Instant scaling
✔ Remote access

---

3. SIMPLE REAL-TIME EXAMPLE

Old method:

Company buys physical server.

Cloud method:

Company rents server online instantly.

---

4. TYPES OF CLOUD DEPLOYMENT MODELS

Very important viva question.

Three main types:

A. PUBLIC CLOUD

Owned by third-party providers.

Example:

- Amazon Web Services
- Microsoft Azure

B. PRIVATE CLOUD

Used only by one organization.

Example:

Bank internal cloud system.

C. HYBRID CLOUD

Combination of public + private cloud.

---

5. CLOUD SERVICE MODELS

Most asked viva topic.

Three major models:

A. IaaS (Infrastructure as a Service)

Provides:

- Virtual servers
- Storage
- Networking

Example:

AWS EC2

B. PaaS (Platform as a Service)

Provides:

- Runtime platform
- Development environment

Example:

Google App Engine

C. SaaS (Software as a Service)

Ready-to-use software online.

Example:

- Gmail
- Google Docs

---

6. DIFFERENCE BETWEEN IaaS, PaaS, SaaS

Model	User Manages
IaaS	OS + Apps
PaaS	Only Apps
SaaS	Just Uses Software

---

7. MAJOR CLOUD PROVIDERS

Top three cloud companies:

---

1. Amazon Web Services

Most popular cloud provider.

---

2. Microsoft Azure

Enterprise-focused cloud.

---

3. Google Cloud

Strong in AI and analytics.

---

8. AWS BASICS FOR DEVOPS

AWS = Amazon Web Services

Popular AWS services:

EC2

Virtual server service.

S3

Storage service.

RDS

Managed databases.

Lambda

Serverless computing.

EKS

Managed Kubernetes service.

---

9. AZURE BASICS

Azure services:

- Virtual Machines
- Azure Blob Storage
- Azure Kubernetes Service (AKS)

---

10. GOOGLE CLOUD BASICS

Google Cloud services:

- Compute Engine
- Cloud Storage
- GKE (Google Kubernetes Engine)

---

11. WHAT IS VIRTUAL MACHINE IN CLOUD?

VM = Software-based computer.

Example:

Cloud server instance.

---

12. WHAT IS AUTO SCALING?

Automatically increase/decrease resources.

Example:

Website traffic rises → more servers added.

---

13. WHAT IS LOAD BALANCER?

Distributes traffic across servers.

Example:

1000 users split among many servers.

---

14. WHAT IS SERVERLESS COMPUTING?

Run code without managing servers.

Example:

AWS Lambda

---

15. CLOUD STORAGE

Stores files online securely.

Examples:

- AWS S3
- Azure Blob Storage

---

16. BENEFITS OF CLOUD COMPUTING

✔ Scalable
✔ Cost-effective
✔ High availability
✔ Global access

---

17. CLOUD IN DEVOPS

Cloud is essential in DevOps because:

DevOps needs:

- Fast deployment
- Instant infrastructure
- Automation

Cloud enables all these.

---

18. DEVOPS + CLOUD RELATIONSHIP

Example flow:

```
GitHub → Jenkins → Docker → Kubernetes → AWS Cloud Deployment
```

GitHub

Jenkins

Docker

Kubernetes

---

19. WHAT IS CLOUD DEPLOYMENT IN CI/CD?

After code testing:

Application automatically deployed to cloud server.

Example:

Jenkins deploys app to AWS EC2.

---

20. REAL-TIME EXAMPLE

Suppose shopping website needs launch.

Without cloud:

Buy servers manually.

With cloud:

Launch server in minutes.

---

21. COMMON CLOUD CHALLENGES

Challenge 1:

Security risks

Challenge 2:

Cost mismanagement

Challenge 3:

Vendor lock-in

---

22. PUBLIC CLOUD VS PRIVATE CLOUD

Public	Private
Shared provider resources	Dedicated resources
Lower cost	Higher security

---

23. CLOUD SECURITY BASICS

Important security measures:

- Encryption
- IAM roles
- Firewalls

---

24. WHAT IS IAM?

IAM = Identity and Access Management

Controls:

Who can access what.

Example:

AWS IAM users.

---

25. WHAT IS REGION IN CLOUD?

Region = Physical geographic data center location.

Example:

Mumbai region.

---

26. WHAT IS AVAILABILITY ZONE?

Separate isolated data centers inside region.

Used for redundancy.

---

27. IMPORTANT VIVA QUESTIONS

Q1: What is cloud computing?

Providing computing resources over internet.

Q2: What is IaaS?

Infrastructure as a Service.

Q3: Difference SaaS vs PaaS?

SaaS gives ready software,
PaaS gives development platform.

Q4: Name top cloud providers.

AWS, Azure, Google Cloud.

Q5: Why cloud important in DevOps?

Enables scalable automated deployments.

---

28. TRICKY EXAMINER QUESTIONS

"Is cloud always public internet?"

No.
Private cloud exists too.

“Can cloud exist without internet?”

Internal private cloud can.

“Does cloud mean no physical servers?”

Wrong.
Cloud still uses physical servers in data centers.

---

29. INTERVIEW BONUS QUESTION

“Why AWS dominates cloud market?”

Because:

- Early market entry
- Huge service ecosystem

---

30. REAL COMPANY USAGE

Cloud heavily used by:

- Netflix
- Airbnb
- Uber

---

31. MEMORY SHORTCUT

Remember:

Cloud = Rent IT Resources On Demand

---

32. SUMMARY IN ONE LINE

Cloud computing provides scalable on-demand computing resources over the internet for modern DevOps systems.

When ready type:

NEXT

Then we continue with:

TOPIC 12: MONITORING TOOLS IN DEVOPS COMPLETE OVERVIEW

Including:

- Monitoring basics
- Prometheus
- Grafana
- Logging
- Alerting
- DevOps observability

09/04/2026, 14:00:37

- Full viva questions